

ACADEMIC MINOR PROJECT REPORT

DESIGN AND DEVELOPMENT OF AN AI-POWERED ONLINE EXAMINATION & STUDENT PERFORMANCE ANALYSIS PLATFORM (SMARTEXAMS)

A Minor Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING / ARTIFICIAL INTELLIGENCE

Submitted by:

- Student Name: [Student Name]
- PRN / Roll No.: [PRN / Roll No]
- Academic Year: 2025 – 2026
- Semester: IV / VI (Second / Third Year Minor Project)

Under the Guidance of:

- Project Guide: Prof. [Guide Name]
- Department: Department of Computer Science & Engineering / Artificial Intelligence
- Institution: G H Rasoni College of Engineering and Management (GHRCEM), Jalgaon
- Affiliated University: KBC North Maharashtra University / Autonomous

CERTIFICATE OF APPROVAL

This is to certify that the Minor Project Report entitled "**DESIGN AND DEVELOPMENT OF AN AI-POWERED ONLINE EXAMINATION & STUDENT PERFORMANCE ANALYSIS PLATFORM (SMARTEXAMS)**" is a bonafide work carried out by **[Student Name] (Roll No / PRN: [PRN / Roll No])** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering / Artificial Intelligence at G H Raison College of Engineering and Management, Jalgaon, during the academic session 2025–2026.

The project has been examined, evaluated, and approved by the undersigned board of examiners.

Internal Examiner Date:

External Examiner Date:

Project Guide
Department of CSE/AI GHRCEM

Head of Department (CSE/AI)
GHRCEM, Jalgaon

DECLARATION

I hereby declare that this minor project report entitled "**DESIGN AND DEVELOPMENT OF AN AI-POWERED ONLINE EXAMINATION & STUDENT PERFORMANCE ANALYSIS PLATFORM (SMARTEXAMS)**" is my original work. It does not contain any material previously submitted for the award of any other degree or diploma in this or any other university.

All quotations, methodologies, libraries, design patterns, and source code utilized in this software development effort have been duly credited and cited in the references.

Date: October 2026

Place: Jalgaon, Maharashtra

Student Name: [Student Name]

PRN / Roll No: [PRN / Roll No]

Department: Department of Computer Science & Engineering

ABSTRACT

SmartExams is a full-stack AI-powered online examination and student performance analysis platform developed for B.Tech Computer Science and Artificial Intelligence education. Engineered with React 18, Vite 8, Redux Toolkit, and Axios on the frontend, and Python 3.14, Django 5 REST Framework, SimpleJWT, and SQLite on the backend, the system delivers secure role-based e-assessment, server-synced exam engine sessions, hybrid AI short-answer evaluation (Gemini 2.5 Flash API + deterministic N-gram keyword fallback), browser proctoring audit tracking, and real-time diagnostic performance analytics.

CHAPTER 1: INTRODUCTION

1.1 Context and Background

In modern software engineering and computer science education, digital assessment platforms have become indispensable infrastructure for evaluating academic competencies. Traditional paper-based examinations and generic web forms (such as Google Forms or basic survey software) present severe structural drawbacks: high administrative grading overhead, vulnerability to academic dishonesty, inability to grade subjective short-answer responses semantically, and absence of instantaneous feedback loops for students.

1.2 Problem Statement

Problem Statement: To design, implement, and validate a full-stack e-examination platform (SmartExams) featuring role-based access control, interactive exam engines with server-synced countdown timers and answer autosave, proctoring violation monitoring, hybrid AI short-answer semantic evaluation with deterministic keyword fallback, and real-time student performance analytics using React 18, Redux Toolkit, Django REST Framework, and SQLite.

CHAPTER 4: SYSTEM ARCHITECTURE & DIAGRAMS

Figure 4.1: SmartExams Decoupled Client-Server System Architecture

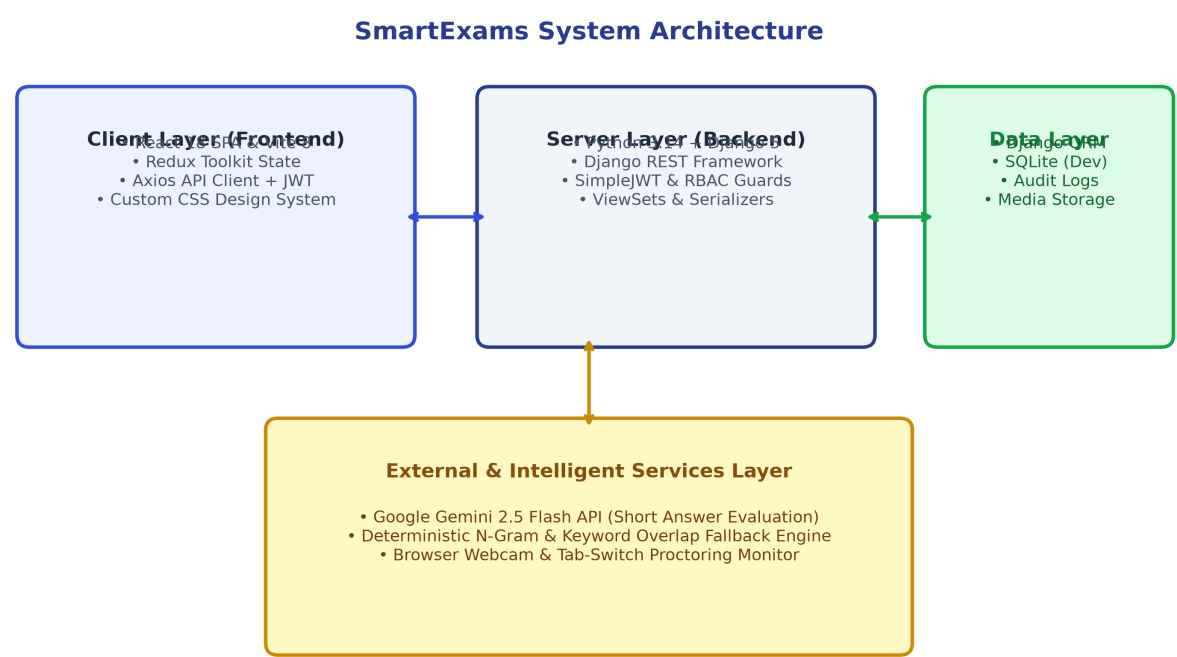


Figure 4.2: SmartExams Unified Use Case Diagram

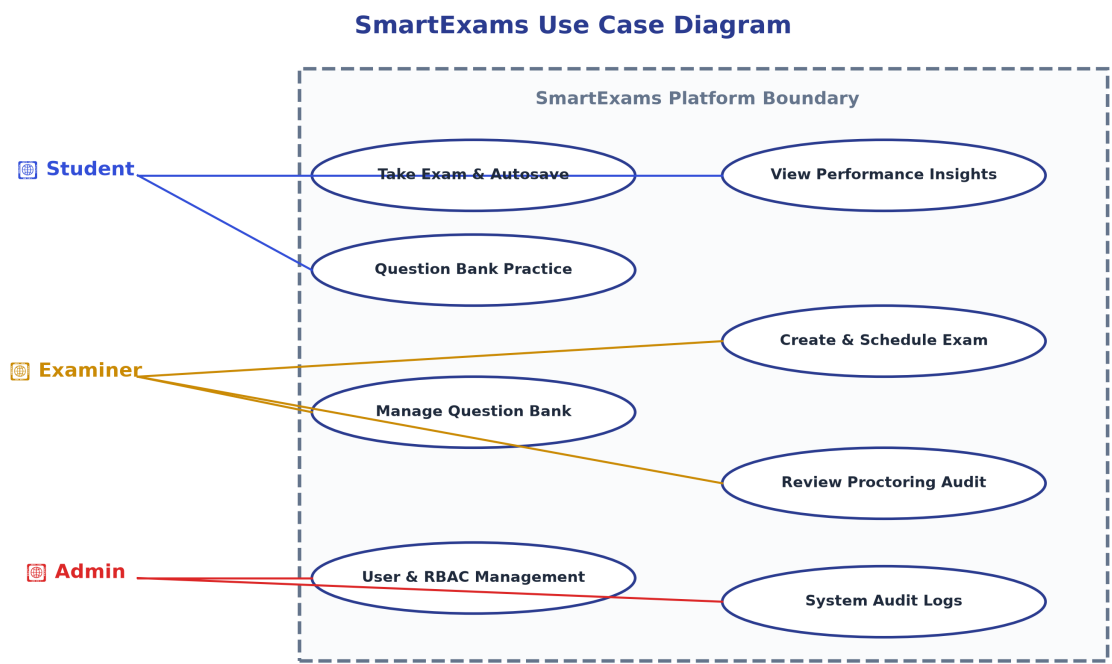


Figure 4.3: Data Flow Diagram (DFD Level 1)

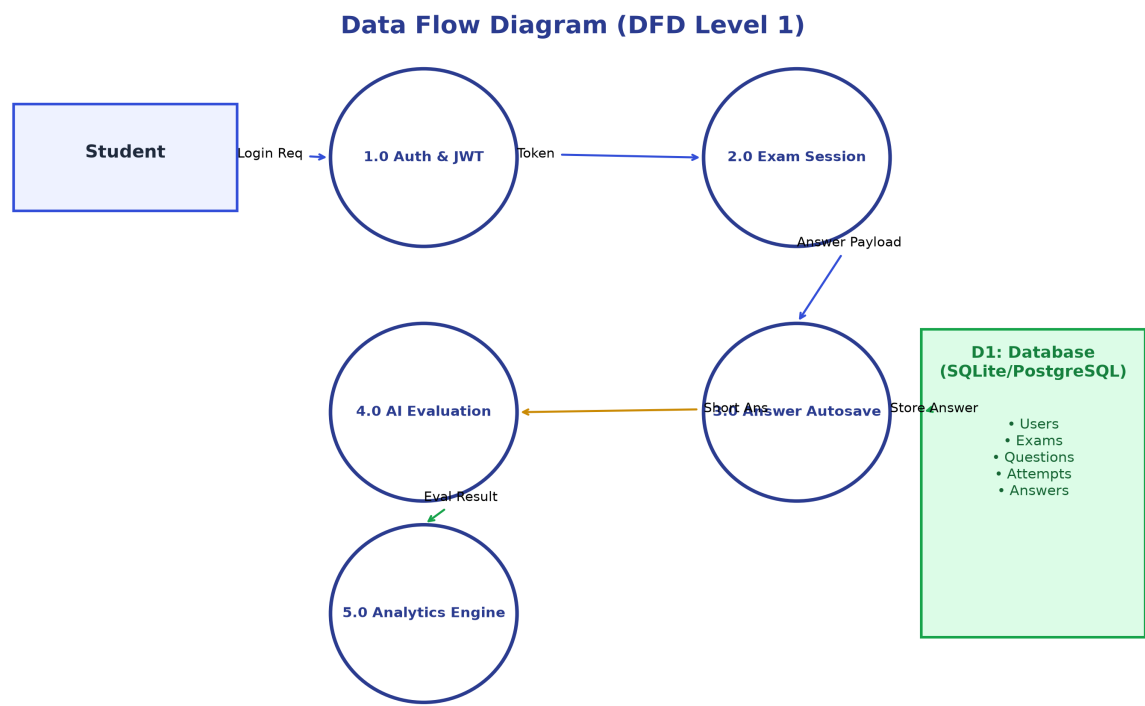


Figure 4.4: Database Entity-Relationship (ER) Schema

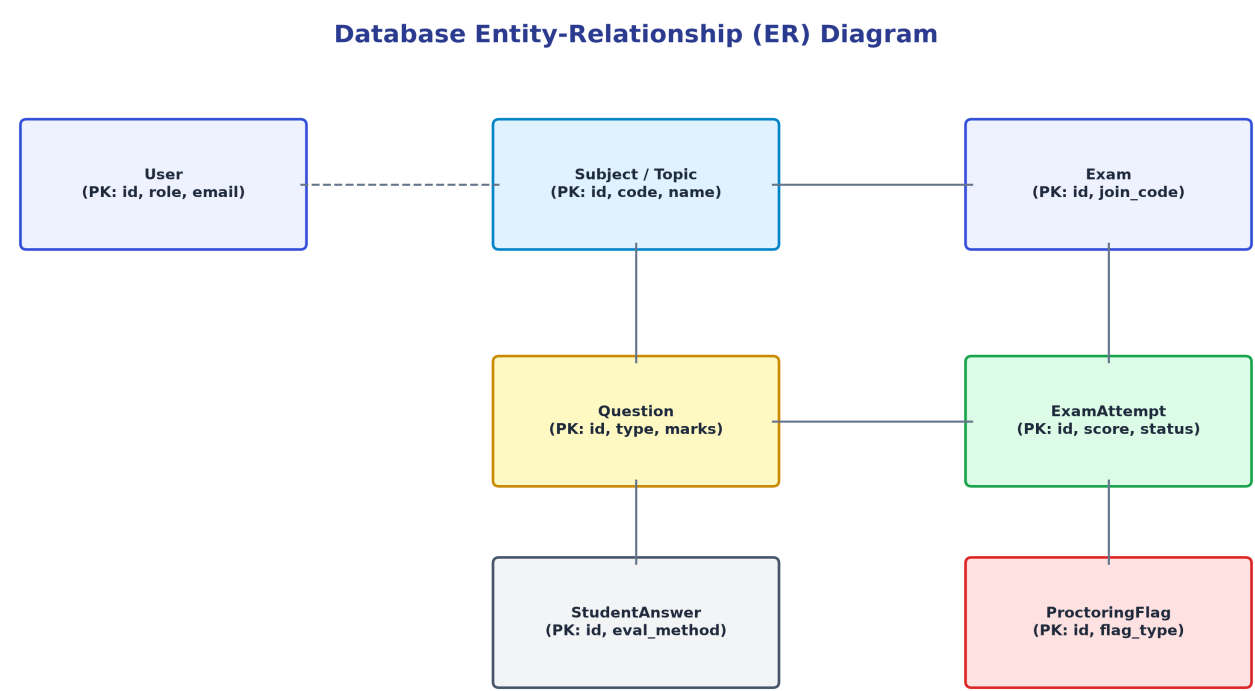


Figure 4.5: Hybrid AI Short-Answer Evaluation Workflow

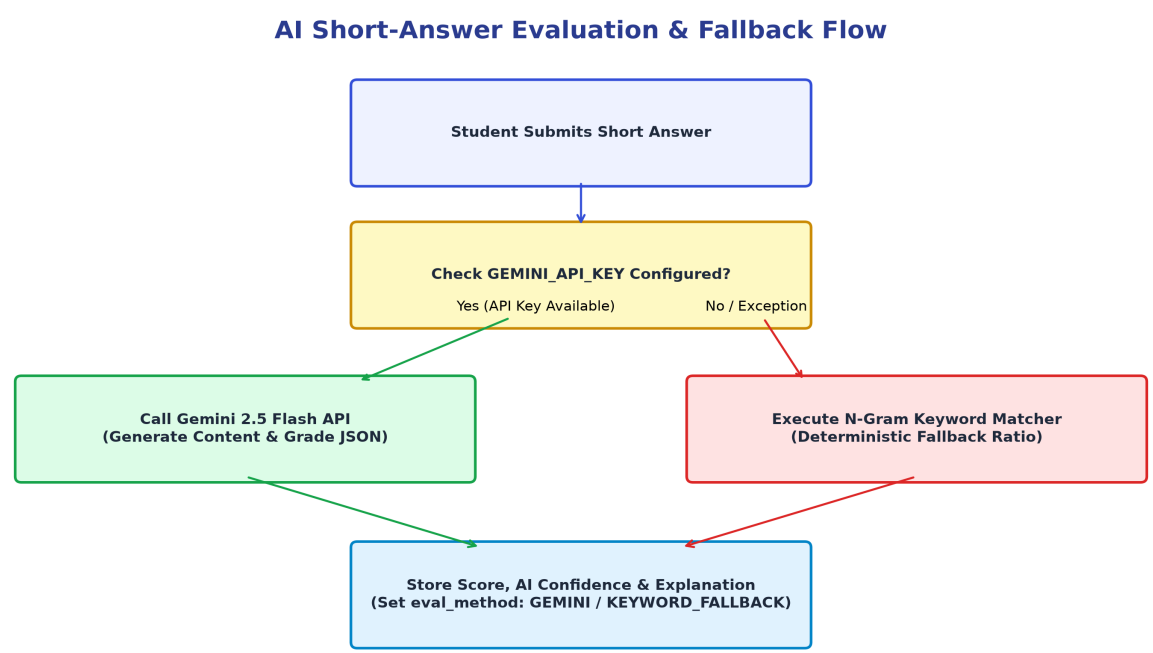
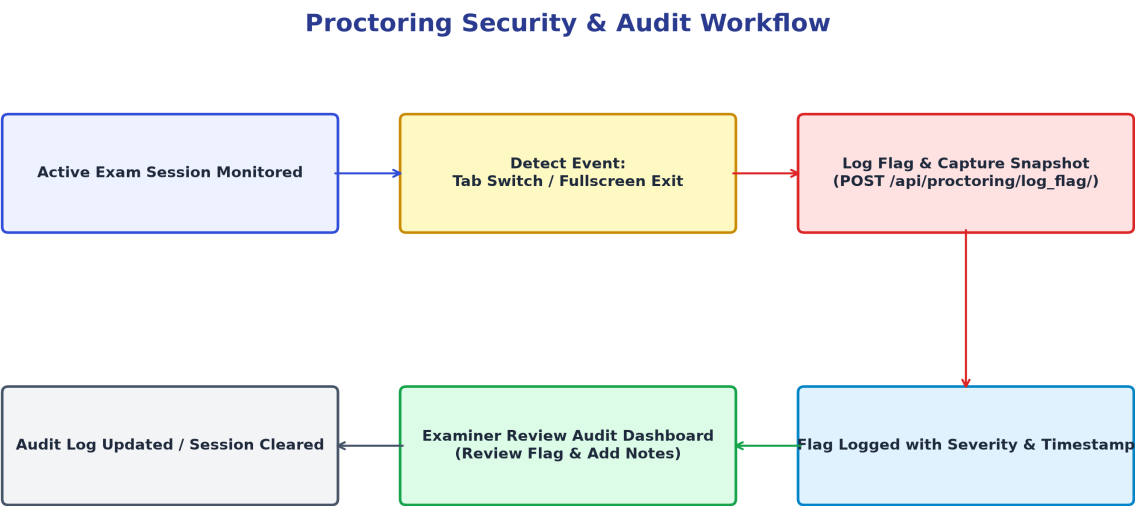


Figure 4.6: Proctoring Security & Flag Review Workflow



CHAPTER 9: PRESENTATION & VIVA DEFENSE PREPARATION (35 Q&A;)

Q1: What is the main objective of the SmartExams platform?

Model Answer: "To provide a secure, full-stack e-examination system with role-based access control, server-synced test engines, hybrid AI short-answer evaluation, proctoring violation tracking, and instant student performance analytics."

Q2: What tech stack is used in SmartExams?

Model Answer: "Frontend: React 18, Vite 8, Redux Toolkit, Axios, Lucide Icons, Custom CSS. Backend: Python 3.14, Django 5, DRF, SimpleJWT, SQLite."

Q3: How is role-based access control (RBAC) enforced?

Model Answer: "Enforced on both frontend React Router guards (``) and backend DRF permission classes (``IsStudent``, ``IsExaminer``, ``IsAdminUser``)."

Q4: Can a user self-register as an Admin?

Model Answer: "No. ``RegisterSerializer`` in Django backend explicitly validates role input and returns HTTP 400 Bad Request if role='ADMIN' is submitted."

Q5: How does SimpleJWT authentication work in your system?

Model Answer: "Users authenticate via ``/api/auth/token/`` to receive access and refresh token pairs. Axios request interceptor attaches the access token as a Bearer header."

Q6: How does token refresh work when an access token expires?

Model Answer: "Axios response interceptor catches HTTP 401, calls ``/api/auth/token/refresh/`` using the stored refresh token, updates localStorage, and retries the original API request."

Q7: How does answer autosave work during an active exam?

Model Answer: "Every option selection or short answer text edit triggers a background ``POST /api/attempts/:id/save\_answer/`` request, persisting data immediately to ``StudentAnswer`` table."

Q8: How does the system handle browser refreshes during an exam?

Model Answer: "The engine calls ``GET /api/attempts/:id/attempt\_state/``, restoring remaining timer seconds, question palette states, and saved answers without creating a new attempt."

Q9: How is the countdown timer synchronized?

Model Answer: "Remaining time is computed on the server: ``deadline = start\_time + duration``. Frontend calculates remaining seconds from this server deadline."

Q10: What happens when the countdown timer reaches zero?

Model Answer: "The engine automatically executes ``handleAutoSubmit()`` , calling the backend submit endpoint which evaluates answers and marks attempt as ``EVALUATED``."

Q11: How are short-answer questions evaluated?

Model Answer: "Evaluated using a hybrid service: first calls Google Gemini 2.5 Flash API. If API key is missing or fails, it falls back to a deterministic N-gram & keyword overlap algorithm."

Q12: Does the system break if the Gemini API key is unconfigured?

Model Answer: "No. The `ai\_evaluator.py` service catches exceptions and seamlessly routes evaluation to the deterministic keyword matcher (`KEYWORD\_FALLBACK`)."

Q13: What evaluation fields are stored for each student answer?

Model Answer: "Stores `is\_correct`, `score\_obtained`, `ai\_confidence`, `ai\_explanation`, `evaluation\_method`, and `examiner\_override`."

Q14: How are MCQ and True/False questions graded?

Model Answer: "Automatically evaluated on backend by comparing student choice with `Question.correct\_answer`, applying positive marks or negative marking deductions."

Q15: How does proctoring violation detection work?

Model Answer: "Browser window blur, tab switch, and fullscreen exit event listeners send `POST /api/proctoring/log\_flag/` logging severity, timestamp, and snapshot."